AI Observability in OpenClaw - Study Guide

■ What is AI Observability?

Observability for AI systems means being able to answer: - **What happened?** - Trace every request through the system - **How much did it cost?** - Track token usage and API costs - **How well did it work?** - Measure quality, latency, errors - **Where are the bottlenecks?** - Identify performance issues

■ OpenClaw's Built-in Observability

1. Session Transcripts (Basic Logging)

Location: `src/config/sessions/store.ts`

```
// Every session is logged to JSONL files
// Format: sessions/{agent}/{scope}/{sessionKey}/transcript.jsonl
```

What it captures: - Every message (user + assistant) - Tool calls and results - Timestamps - Token counts - Model used

File structure:

```
sessions/
■■■ agent:main:main/
   ■■■ transcript.jsonl    # One line per message/tool call
```

How to read:

```
# View raw transcript
cat sessions/agent:main:main/transcript.jsonl | jq
```

```
# Count total messages
wc -l sessions/agent:main:main/transcript.jsonl

# Extract tool calls
grep '"type":"tool_use"' sessions/agent:main:main/transcript.jsonl
```

2. Agent Metrics (Runtime Stats)

Location: `src/agents/pi-embedded-runner/run.ts` (line ~350-400)

```
// Token tracking happens during execution
const usage = {
 inputTokens: response.usage.input_tokens,
 outputTokens: response.usage.output_tokens,
 cacheReadTokens: response.usage.cache_read_input_tokens || 0
}
```

Metrics collected: - Input tokens - Output tokens
- Cache read tokens (prompt caching) - Cache creation tokens - Total cost (calculated from provider pricing)

Where it's stored: - Session state (`sessionState.usage`) - Not persisted to database by default - Available via `/status` command or `session_status` tool

3. Model Provider Tracking

Location: `src/agents/pi-embedded-runner/run.ts` (line ~200-250)

Failover logic:

```
// Primary model attempt
try {
 response = await callModel(primaryModel)
} catch (error) {
 // Failover to backup
 if (isAuthError || isRateLimitError) {
   response = await callModel(fallbackModel)
 }
}
```

What you can track: - Which model was actually used - Failover events (auth failures, rate limits) - Auth profile rotation - Context overflow handling

4. Compaction Events

Location: `src/agents/context.ts` (line ~100-150)

Why it matters: Compaction = losing detailed history

```
// Triggered when context exceeds limits
if (contextSize > maxContextSize * 0.9) {
 // Summarize old messages
 const summary = await generateSummary(oldMessages)
 // Replace with compact version
}
```

Observable via: - Session status shows compaction count - `COMPACTION` events in transcript - Memory flush writes to `memory/` files (if enabled)

■ What OpenClaw DOESN'T Have (Yet)

Missing Observability Features:

1. **No centralized dashboard** - Each session is isolated JSONL
2. **No cost aggregation** - Can't see total spend across sessions
3. **No performance metrics** - Latency not tracked systematically
4. **No error rates** - No automatic error counting/alerting
5. **No user analytics** - Can't track user behavior patterns
6. **No A/B testing** - Can't compare prompt/model variations
7. **No quality metrics** - No automated evaluation scores

■■ How to Add External Observability

Option 1: Langfuse Integration

What Langfuse provides: - Centralized trace storage - Cost analytics dashboard - Latency metrics - Error tracking - User sessions - Prompt versioning - LLM evaluation scores

How to integrate with OpenClaw:

```
// src/agents/pi-embedded-runner/run.ts
import { Langfuse } from 'langfuse'

const langfuse = new Langfuse({
 publicKey: process.env.LANGFUSE_PUBLIC_KEY,
 secretKey: process.env.LANGFUSE_SECRET_KEY
})

// Wrap each agent call
const trace = langfuse.trace({
 name: 'agent-run',
 sessionId: sessionKey,
 userId: deliveryContext.senderId,
 metadata: {
   model: modelId,
   channel: deliveryContext.channel
 }
})

// Track generation
const generation = trace.generation({
 name: 'llm-call',
 model: modelId,
 input: messages,
 startTime: new Date()
})

// After completion
generation.end({
 output: response.content,
 usage: {
   input: response.usage.input_tokens,
   output: response.usage.output_tokens
 },
 endTime: new Date()
})
```

Option 2: Custom Logging to DB

Simple approach - Log to PostgreSQL:

```
CREATE TABLE agent_runs (
 id SERIAL PRIMARY KEY,
 session_key TEXT,
 model TEXT,
 input_tokens INT,
 output_tokens INT,
 cache_read_tokens INT,
 cost_usd DECIMAL(10,6),
 latency_ms INT,
 error TEXT,
 created_at TIMESTAMP DEFAULT NOW()
);
```

Insert after each run:

```
// src/agents/pi-embedded-runner/run.ts (end of run)
await db.query(`
 INSERT INTO agent_runs
 (session_key, model, input_tokens, output_tokens, cost_usd, latency_ms)
 VALUES ($1, $2, $3, $4, $5, $6)
`, [sessionKey, modelId, usage.input, usage.output, cost, latency])
```

■ Metrics to Track

Essential Metrics:

1. **Token Usage**
2. Input tokens per session
3. Output tokens per session

Cache hit rate (cache_read / total_input)

Costs

6. Cost per session
7. Cost per day/week/month

8. Cost by model

Cost by user/channel

Performance

11. End-to-end latency (request → response)
12. Token generation speed (tokens/second)

Time to first token

Reliability

15. Error rate (failed calls / total calls)
16. Failover rate (fallback model usage)

Retry count

Quality

19. User feedback (if collected)
20. Tool call success rate
21. Conversation completion rate

OpenClaw-Specific Metrics:

- **Compaction frequency** - How often do sessions hit context limits?
- **Model distribution** - What % use primary vs fallback models?
- **Channel breakdown** - Usage by WhatsApp/Telegram/Discord/etc
- **Agent type** - Main sessions vs spawned sub-agents
- **Cron job metrics** - Heartbeat execution time, wake event frequency

■ Study Exercises

Exercise 1: Parse Session Transcript

```
# Load transcript
cat sessions/agent:main:main/transcript.jsonl | jq -s '
 # Calculate total tokens
 map(select(.usage != null)) |
 map(.usage.input_tokens + .usage.output_tokens) |
 add
'
```

Exercise 2: Build Cost Calculator

```
# Read OpenClaw transcripts and calculate costs
import json

# Pricing (as of 2026)
PRICING = {
   'anthropic/claude-sonnet-4-5': {
       'input': 3.00 / 1_000_000,   # $3 per 1M
       'output': 15.00 / 1_000_000  # $15 per 1M
   }
}

total_cost = 0
with open('sessions/agent:main:main/transcript.jsonl') as f:
   for line in f:
       msg = json.loads(line)
       if 'usage' in msg:
           model = msg.get('model', 'anthropic/claude-sonnet-4-5')
           cost = (
               msg['usage']['input_tokens'] * PRICING[model]['input'] +
               msg['usage']['output_tokens'] * PRICING[model]['output']
           )
           total_cost += cost

print(f"Total cost: ${total_cost:.4f}")
```

Exercise 3: Track Failover Events

```
# Find all model failover events in logs
grep -r "Failover" sessions/ | wc -l
```

```
# Check which models are actually being used
cat sessions/*/transcript.jsonl | \
 jq -r 'select(.model != null) | .model' | \
 sort | uniq -c
```

■ Further Reading

- **OpenClaw Agent Runtime:** `\openclaw\src\agents\pi-embedded-runner\run.ts`
- **Session Storage:** `\openclaw\src\config\sessions\store.ts`
- **Model Config:** `\openclaw\src\agents\model.ts`
- **Langfuse Docs:** https://langfuse.com/docs
- **OpenTelemetry:** https://opentelemetry.io/ (industry standard)

■ Next Steps

1. ■ Understand where OpenClaw already logs data
2. ■ Choose observability platform (Langfuse, LangSmith, or custom)
3. ■ Instrument agent runtime with tracing
4. ■ Build cost dashboard
5. ■ Set up alerts for anomalies